1.

R4 7, 11, 18, 29, 47

R5 4, 7, 11, 18, 29

R6 4, 3, 2, 1, 0

2.

```
        MOV R3, #1
        MOV R1, #0
loop    TST R0, R3
        ADDNE R1, R1, #1
        ADDS R3, R3, R3
        BNE loop
```

3.

```
         MOV R2, #1
loop     MUL R2, R2, R0
         SUBS R1, R1, #1
         BNE loop
```

4.

```
       MOV R2, #0
       CMP R1, #0
       BEQ halt
loop:  TST R1, #1
       ADDNE R2, R2, R0
       MOV R0, R0, LSL #1
       MOVS R1, R1, ASR #1
       BNE loop
halt:  B halt
```

5.

[Technically, PC would always be 4 more than listed below. However, the below answer is more intuitive and is acceptable, as long as you're consistent.]

PC: 0, 4, 8, 12, 16, 20, 24, 8, 12, 16, 20, 24, 8, 12, 16, 20, 28, 32

R0: 0, 5, 9, 12

R1: 5, 4, 3, 2

R2: 1

6.

It looks at the Z flag to see whether the Z flag is 0 or 1. If the Z flag is 1, then it changes R15 (the program counter) to the address named within the instruction. If the Z flag is 0, then it increases R15by 4 (so that the next instruction executed is the next instruction after the **BEQ** instruction).

7.

```
        SUB R2, R0, #4      ; R2 will be address of first zero found
again   LDR R1, [R2, #4]!
        CMP R1, #0
        BNE again
        SUB R0, R2, R0      ; now compute index corresponding to R2
        MOV R0, R0, ASR #2  ; by subtracting R0 and dividing by 4
```

- **[R2, #4]!:** The addressing mode specifies the memory location to load the word from. The address is calculated as the value in register R2 plus an offset of 4 (i.e., the fourth byte after the address in R2). The exclamation mark at the end of the addressing mode operand indicates that the value in R2 should be updated to R2 + 4 after the memory access.

- **Important Note:**
  The two assembly instructions **LDR R0, [R2], #4** and **LDR R0, [R2, #4]** both load a 32-bit word from memory into the R0 register. However, there is a subtle difference in how the memory address is computed.
- The **LDR R0, [R2], #4** instruction loads a word from memory at the address stored in R2 and then increments R2 by 4 bytes. This means that it loads the word at the current address stored in R2, and then updates R2 to point to the next word in the memory.
- On the other hand, the **LDR R0, [R2, #4]** instruction loads a word from memory at the address calculated by adding 4 bytes to the value in R2. This means that it loads the word from the memory location that is 4 bytes after the address stored in R2, but does not modify the value of R2.

8.

```
        MOV R2, R0          ; R2 is address of next entry to read
        LDR R0, [R2], #4    ; Initialize return value with first entry
        B start
next    LDR R3, [R2], #4    ; Retrieve next entry
        CMP R3, R0          ; Update R0 if this entry is larger
        MOVGT R0, R3
start   SUBS R1, R1, #1     ; Test whether any entries are left
        BNE next
```

- **MOV R2, R0**: This moves the value in register R0 into register R2. In this context, R0 likely contains the address of the beginning of the array, so R2 will be used to keep track of the current position in the array as we iterate over it.
- **LDR R0, [R2], #4**: This loads the first entry in the array into register R0, and then increments R2 by 4 bytes (since we assume each entry is a 32-bit integer).
- **B start**: This jumps to the **start** label, which we will define later.
- **next LDR R3, [R2], #4**: This loads the next entry in the array into register R3, and then increments R2 by 4 bytes.
- **CMP R3, R0**: This compares the value in R3 to the value in R0.
- **MOVGT R0, R3**: This moves the value in R3 into R0 if the value in R3 is greater than the value in R0.
- **start SUBS R1, R1, #1**: This subtracts 1 from the value in R1, which likely contains the number of entries in the array that still need to be checked.

- **BNE next**: This jumps to the **next** label if the result of the subtraction in the previous line is not equal to zero. In other words, this loops back to step 4 until we have checked every entry in the array.

9.

```
        MOV R3, #1
        MOV R2, #1
loop    CMP R2, R1
        BGE done
        LDR R4, [R0, R2, LSL #2]
        SUB R5, R2, #1
        LDR R5, [R0, R5, LSL, #2]
        CMP R4, R5
        STRNE R4, [R0, R3, LSL #2]
        ADDNE R3, R3, #1
        ADD R2, R2, #1
        B loop
done    MOV R1, R3
```

To understand more "**STRNE R4, [R0, R3, LSL #2]"** instruction:

The instruction **STRNE R4, [R0, R3, LSL #2]** stores the value in register R4 into memory at the address given by the sum of the value in R0, the value in R3 shifted left by 2 bits, and an optional condition code.
Here's what each operand means:

- **STRNE**: This is the mnemonic for the "store register" instruction, which stores the value in a register to memory.
- **R4**: This is the register containing the value to be stored in memory.
- **[R0, R3, LSL #2]**: This is the memory address where the value in R4 will be stored. It is calculated as the sum of the value in R0 (which holds the base address of the array), the value in R3 shifted left by 2 bits (which multiplies R3 by 4 to convert it from an array index to a byte offset), and an optional condition code (in this case, "NE" for "not equal", which means the store operation will only be executed if the previous comparison instruction indicated that the previous element was greater than the current element). The square brackets indicate that this is a memory access operation, rather than a register-to-register operation.

10.

```
LDR R3, [R0, R2, LSL #2]
LDR R3, [R3, #4]
```

To understand more "**LDR R3, [R0, R2, LSL #2]"** instruction:

- "**[R0, R2, LSL #2]**" is the memory address where the value to be loaded is located. This is an expression that calculates the address using three components:
    - "**R0**" is the base register, which holds the base address of the memory region being accessed.
    - "**R2**" is the offset register, which holds the offset value to be added to the base address.
    - "**LSL #2**" is a shift operation that multiplies the offset value by 4, which is equivalent to shifting it left by 2 bits. This is because ARM instructions are

word-aligned, meaning that they operate on 32-bit words, so the offset must be multiplied by 4 to get the correct byte offset.

Therefore, the final memory address used in this instruction is calculated as: **R0 + (R2 << 2).**

11.

```
     MOV R1, #0          # no negative numbers found so far
loop TST R0, R0          # see if R0 is NULL
     BEQ done
     LDR R2, [R0]        # load R0->data into R2
     CMP R2, #0
     ADDLT R1, R1, #1    # increment R1 if R2 < 0
     LDR R0, [R0, #4]    # load R0->next into R0, and repeat
     B loop
done
```

To understand more TST instruction:

- For example, the instruction "**TST R1, #1**" is an ARM assembly language instruction that tests the value in register R1 and a bitmask of 1.
- The TST instruction performs a bitwise AND between the two operands and sets the condition code flags in the processor based on the result. The purpose of this instruction is typically to check whether a particular bit in R1 is set or clear.
- If the bit that corresponds to the 1 in the bitmask is set in R1, the zero flag (Z) in the condition code register is set to 0. Otherwise, if the bit is clear, the Z flag is set to 1. The other condition code flags (N, C, and V) are not affected by this instruction.
- Note that the TST instruction does not modify the contents of R1, it only sets the condition code flags based on the result of the bitwise AND operation.